CSC1006 - Assignment 3

Due Date: 2026.05.08 23:59:59

Instructions:

Submit your work in the form of a single file onto Blackboard before the deadline. our submission should include solutions to the theoretical exercises, all the scripts you made use of for addressing the computer-based exercises, and all outputs from your scrips including the figures. You may use a .zip file if you have multiple files. Late submission within 1 day will receive a 50% penalty. Late submission with >1 day will be rejected.

Problem 1:

Consider a full-connection backpropagating neural network whose structure and the initial weights are shown in Fig. 1. Assume now we have a training data pair with x = 0.5, y = [1.0 0.5]. Please calculate the new weights for all connections after one training epoch with the following given conditions. Calculation process is required.

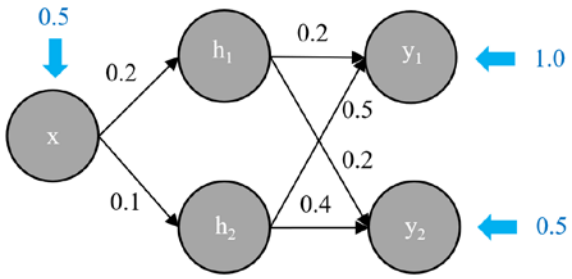

Fig. 1. A 1-2-2 neural network structure.

Learning rate: $\lambda = 0.1$

Activation function: $f(x) = \frac{1}{1+e^{-x}}$

Loss function: $(y_d - y)^2$

Problem 2:

Consider a humanoid robot which is using Q-learning method to learn its walking policy. The robot is supposed to follow a centerline on the floor. Initially, the robot is 15 cm right deviated from the centerline, and its Q-table is shown in Table I. Please use Bellman Equation to update the correct element value in the table after executing one step of action.

Table I: Q-table that stores the action quality value for the humanoid robot

State \ Action	A1 (Step left for 20 cm)	A2 (Step ahead for 20 cm)	A3 (Step right for 20 cm)
S1 (Left: <-10 cm)	0.5	0.2	1
S2 (-10 ~ 10 cm)	2	3	0.5
S3 (Right: >10 cm)	2	0.5	-0.1

We assume a -5, 5, -5 reward for the situations S1, S2 and S3 respectively, and the learning rate being 0.2, discount factor being 0.5. Please elaborate your calculation process.

Problem 3:

Please design a neural network algorithm (NN) with gradient descent weight update rule to learn a math function: $z = (2x - y)^2$. The loss function is designed as the mean squared error (MSE), i.e. $L = (z_d - z)^2$ in this case, where z is the sole output variable. You could design your own network model (number of layers and nodes in each layer), the learning rate, training epoch, etc. The algorithm should output the following:

1) The total loss calculated by the code should be <5.0 (if non-reachable, lower the better);
2) Draw out the 3D layout of the ideal function and the NN-learned function using algorithm-generated figures;

You shall made additional text explanation about your network set-up, learning parameters, and the best final loss you could reach. The code should be submitted with necessary comments in it as well.

Hint: You could use the Lec05 supplementary code from BB as a template to modify.